Reproducible Data Analysis

Chukwubikem Dara

2026-04-21

Table of contents

1 Busiest Airports Analysis

The following analysis examines passenger traffic trends across six of the world's busiest airports — Atlanta, Beijing Daxing, Denver, Dubai, Frankfurt, and Los Angeles — from 2020 to 2025. The data was sourced from Wikipedia's list of busiest airports by passenger traffic.

Table 1: Passenger Traffic at Six of the World's Busiest Airports (2020–2025)

Airport	Location	Year	Total Passengers
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2024	108,067,766
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2025	106,302,208
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2023	104,653,451
Dubai International Airport	Garhoud, Dubai, Dubai	2025	95,200,000
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2022	93,699,630
Dubai International Airport	Garhoud, Dubai, Dubai	2024	92,331,506
Dubai International Airport	Garhoud, Dubai, Dubai	2023	86,994,365
Denver International Airport	Denver, Colorado	2025	82,427,962
Denver International Airport	Denver, Colorado	2024	82,358,744
Denver International Airport	Denver, Colorado	2023	77,837,917
Los Angeles International Airport	Los Angeles, California	2024	76,588,028
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2021	75,704,760
Los Angeles International Airport	Los Angeles, California	2023	75,050,875
Los Angeles International Airport	Los Angeles, California	2025	73,709,594
Denver International Airport	Denver, Colorado	2022	69,286,461
Dubai International Airport	Garhoud, Dubai, Dubai	2022	66,069,981
Los Angeles International Airport	Los Angeles, California	2022	65,924,298
Frankfurt Airport	Frankfurt, Hesse	2025	63,189,666
Frankfurt Airport	Frankfurt, Hesse	2024	61,564,957
Frankfurt Airport	Frankfurt, Hesse	2023	59,355,389
Denver International Airport	Denver, Colorado	2021	58,828,552
Beijing Daxing International Airport	Daxing District, Beijing	2025	53,618,949
Beijing Daxing International Airport	Daxing District, Beijing	2024	49,441,029

Table 1: Passenger Traffic at Six of the World's Busiest Airports (2020–2025)

Airport	Location	Year	Total Passengers
Frankfurt Airport	Frankfurt, Hesse	2022	48,918,482
Los Angeles International Airport	Los Angeles, California	2021	48,007,284
Hartsfield–Jackson Atlanta International Airport	Atlanta, Georgia	2020	42,918,685
Denver International Airport	Denver, Colorado	2020	33,741,129
Dubai International Airport	Garhoud, Dubai	2021	29,110,609
Los Angeles International Airport	Los Angeles, California	2020	28,779,527
Dubai International Airport	Garhoud, Dubai	2020	25,836,771
Beijing Daxing International Airport	Daxing District, Beijing	2021	25,051,012
Frankfurt Airport	Frankfurt, Hesse	2021	24,812,849
Frankfurt Airport	Frankfurt, Hesse	2020	18,768,601

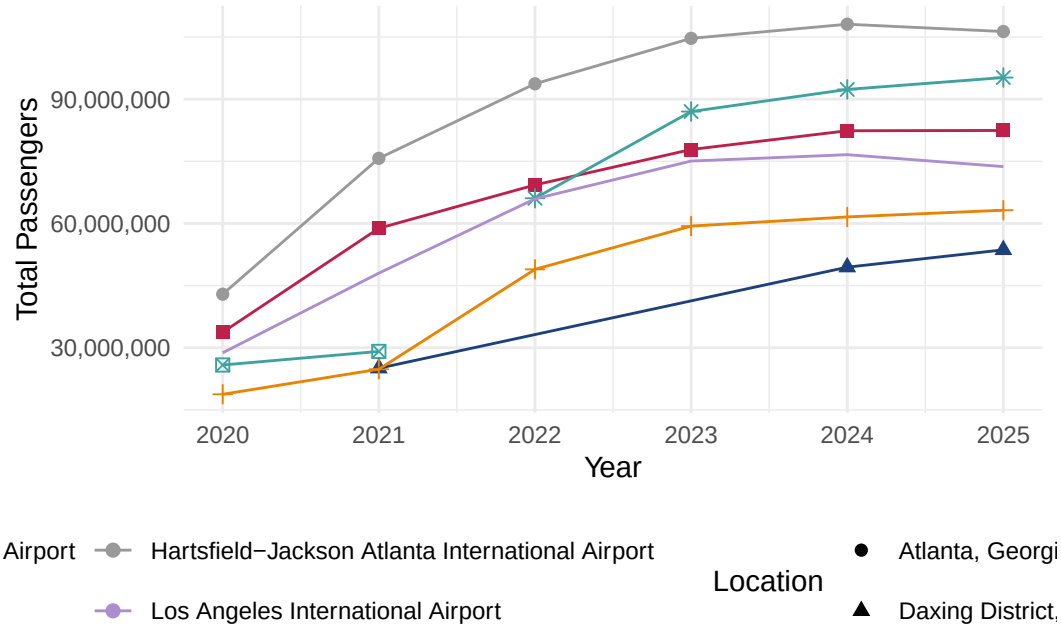

Figure 1: Passenger traffic trends at six major airports from 2020 to 2025

Atlanta's Hartsfield-Jackson consistently leads in total passenger volume across all years,

reflecting its role as a major domestic hub. Dubai and Los Angeles follow closely as key international gateways. All six airports experienced a sharp decline in 2020 and 2021 due to the COVID-19 pandemic, with strong recovery emerging by 2023. Notably, Atlanta and Dubai rebounded faster than Frankfurt and Beijing Daxing, suggesting that domestic-heavy airports recovered more quickly than those reliant on international travel.

2 Monte Carlo Numerical Integration

The Monte Carlo method offers a powerful alternative to traditional deterministic integration techniques. Rather than computing an exact solution analytically, this approach uses random sampling to estimate the area under a curve. Here, I applied this method to the probability density function of the Beta(2, 2) distribution, integrated over the interval [0, 1].

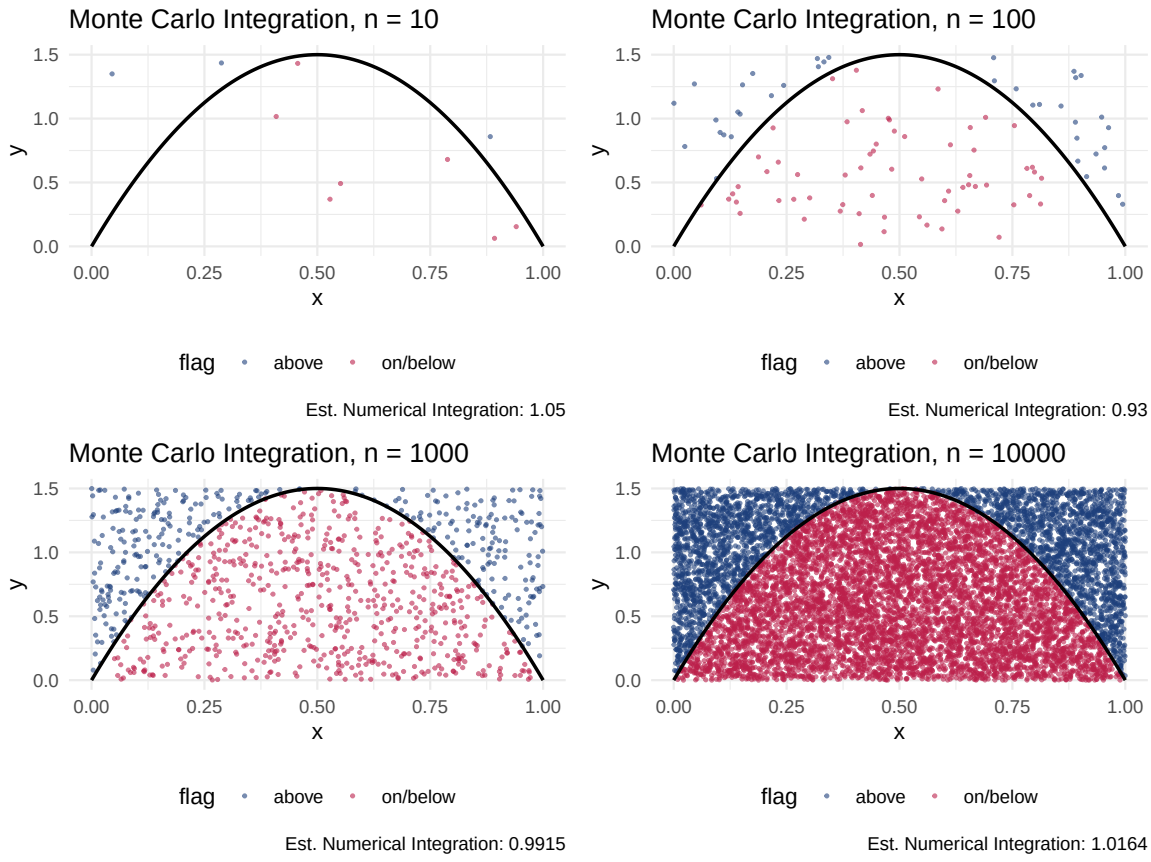

Figure 2: Monte Carlo numerical integration of the Beta(2,2) PDF at four resolutions

As shown in Figure 2, the accuracy of the Monte Carlo estimate improves substantially as the

number of sampled points increases. At n = 10, the estimate is highly variable and unreliable due to the small sample size — the sparse points leave much of the function poorly represented. By n = 100, the estimate begins to stabilize, and at n = 1000 and n = 10000, the estimated integral converges closely to the true value of 1.0. This demonstrates a core principle of Monte Carlo integration: with enough random samples, the proportion of points falling under the curve reliably approximates the true area. The rectangular window used here spans x [0, 1] and y [0, 1.5], giving an area of 1.5, which is multiplied by the proportion to produce each estimate.

3 Planning and Prompting GenAI Tools

3.1 My Plan

Goal: Tidy the calcium data and create a graph to explore the differences in ulnar calcium measurements across time and the two treatment groups.

Needs: Nouns: Calcium Data, Claude, image files, response document, steps (steps to give AI), ggplot, graph Verbs: Tidy, wrangly, clean, mutate, rename, filter, select, arrange, summarize, pivot, separate, geoms, plot

Steps:

1. Load in Calcium Data
2. Rename columns to distinguish the two treatment groups
3. Unite each group's columns into a single column separated by periods
4. Pivot the two treatment columns longer into "treatment" and "values" columns
5. Separate the "values" column on periods back into y0, y1, y2, y3
6. Pivot the time point columns longer into "timePoint" and "measurement" columns
7. Mutate "timePoint" and "treatment" to have clean, readable labels
8. Create a visualization of measurement over time by treatment group
9. Initialize a ggplot with "timePoint" on the x-axis and "measurement" on the y-axis
10. Color by treatment group and add geometry 11.Label axes and apply theme
11. Test and run code

3.2 Results from My Prompt

3.3 Results from Generic Prompt

3.4 Comparison

Both visualizations tell the same general story — bone density stays relatively stable through the first two years before dropping off at Year 3 for both groups. What's interesting is that the

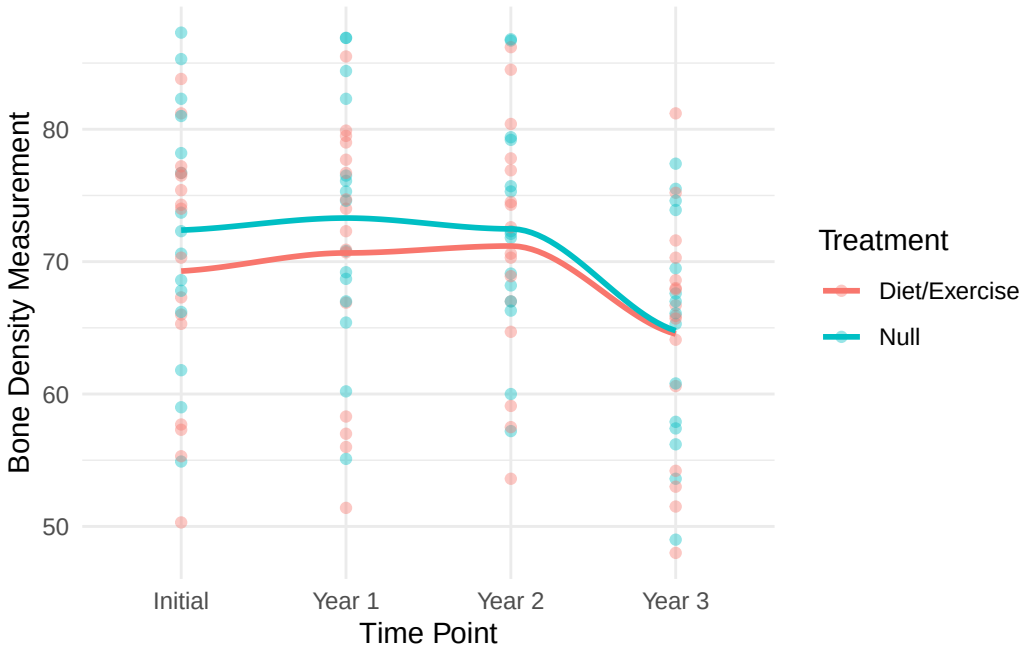

Figure 3: Bone density measurements over time by treatment group

Diet/Exercise group actually starts lower than the Null group but the two groups converge almost completely by Year 3, suggesting the treatment didn't have a dramatic long-term effect on bone density. My plot makes this easier to see because the individual points show just how much variation exists between participants — some people's bone density barely changed while others dropped significantly. The generic prompt's cleaner line plot looks nicer but hides all of that individual variation, which actually tells a more honest story about the data.

4 Reflection

Throughout this course, I've picked up so many tools, methodologies, and honestly new ways of thinking. This PCIP method has given me a new way to tackle how I code and how I face challenges as a whole. Working with Rstudio more and more has given me new insight on how useful it can be for working with large datasets, cleaning, tidying, and creating plots to draw new conclusions. I've worked with built in datasets like penguins and pirates, but also scraped data from the web like the Penn states Womens basketball team or even the busiest airports data. I've learned more than I would've thought throughout Stat 184, and am grateful for the experience and the tools gained.

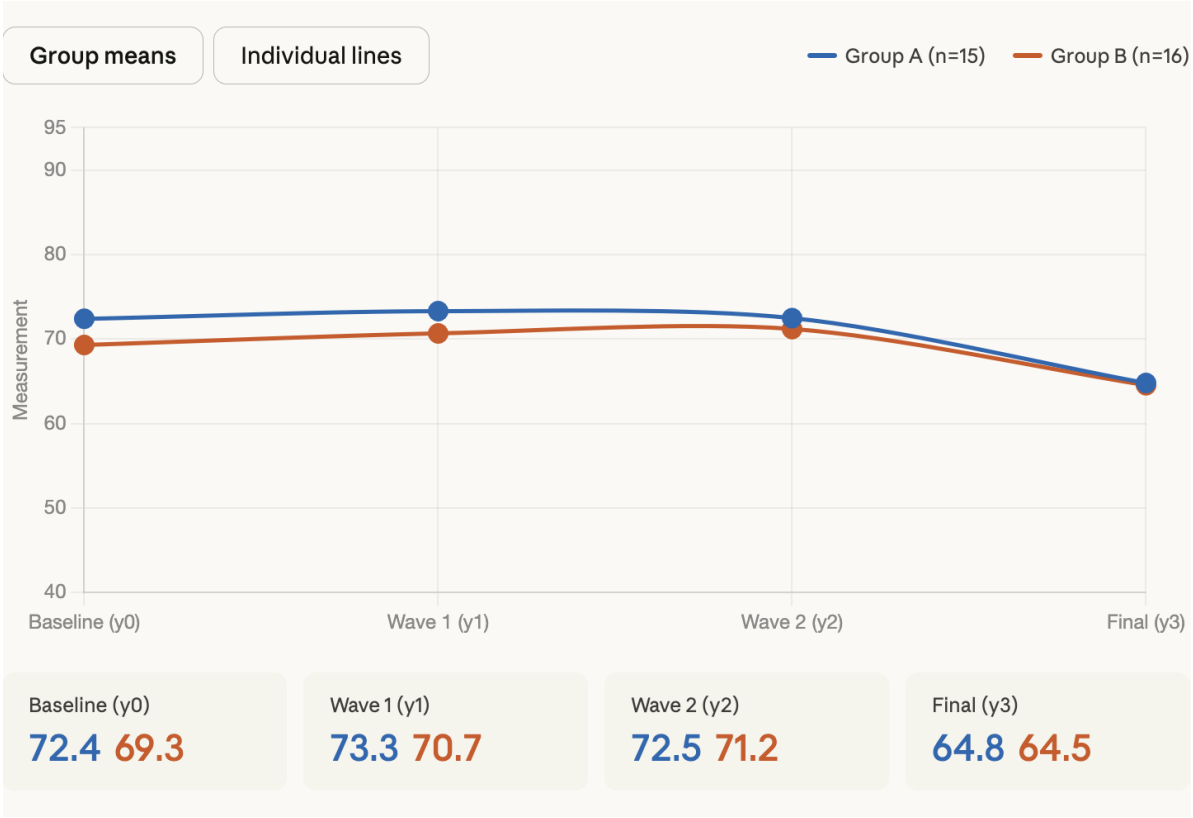

Figure 4: Generic prompt result - Group means view

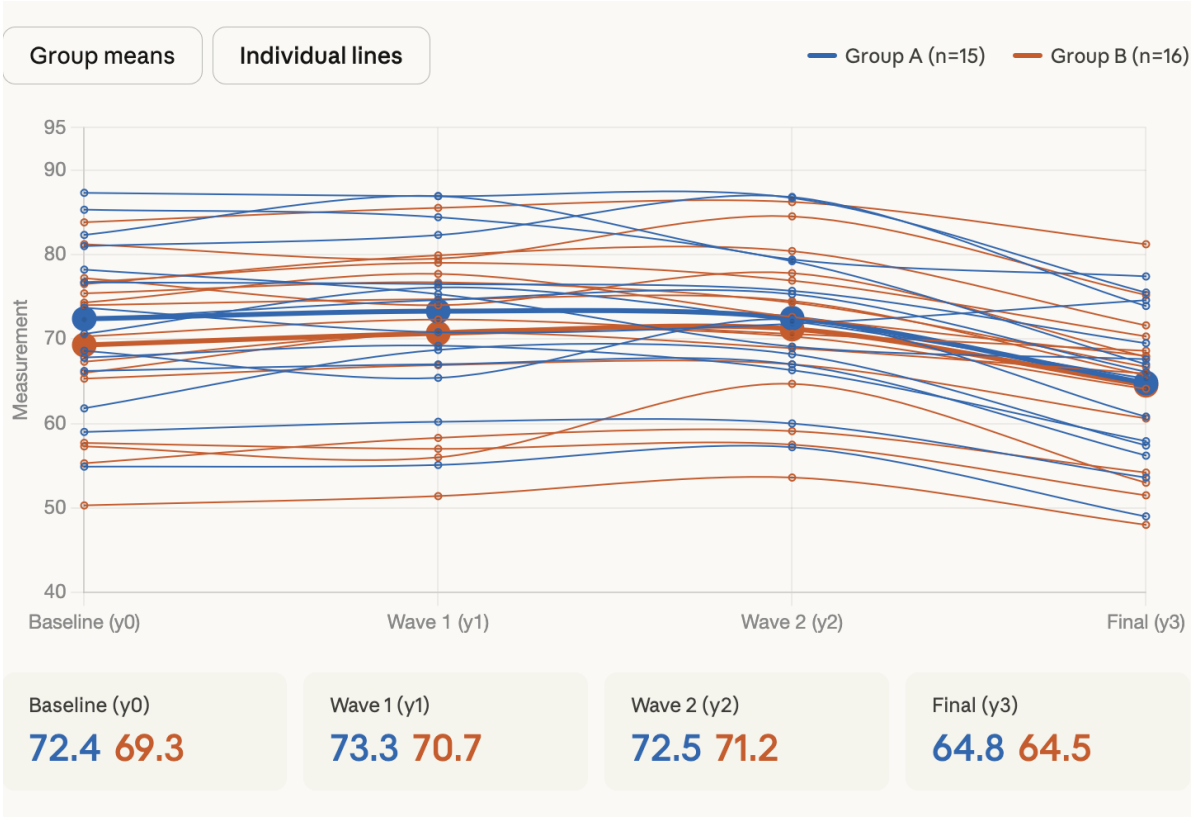

Figure 5: Generic prompt result - Individual lines view

5 Appendix I: GenAI Usage

Tool used: (Anthropic Claude)

Prompt 1:

Paste your exact prompt here.

Response 1:

Paste the exact response here.

6 Appendix II: Code Appendix

```
suppressPackageStartupMessages(library(tidyverse))
suppressPackageStartupMessages(library(ggplot2))
suppressPackageStartupMessages(library(rvest))
library(scales)
library(knitr)
airportsRaw <- read_html(
  x = "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"
) |> html_elements(css = "table") |>
  html_table()

airportsSemiClean <- bind_rows(
  airportsRaw[[1]] |> mutate(year = 2025),
  airportsRaw[[2]] |> mutate(year = 2024),
  airportsRaw[[3]] |> mutate(year = 2023),
  airportsRaw[[4]] |> mutate(year = 2022),
  airportsRaw[[5]] |> mutate(year = 2021),
  airportsRaw[[6]] |> mutate(year = 2020)
) |> filter(str_detect(Airport, "Atlanta|Frankfurt|Beijing Daxing|Denver|Los Angeles|Dubai"))

airportsClean <- airportsSemiClean |> rename(
  TotalPassengers = Totalpassengers,
  RankChange = Rankchange,
  PercentChange = `%change`,
  Year = year
) |> mutate(
  TotalPassengers = readr::parse_number(TotalPassengers)
) |> arrange(desc(TotalPassengers))

psuPallete <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300", "#999999", "#AC8DCE")
airportsClean |>
  select(Airport, Location, Year, TotalPassengers) |>
  mutate(TotalPassengers = scales::comma(TotalPassengers)) |>
  rename("Total Passengers" = TotalPassengers) |>
  knitr::kable()
ggplot(
  data = airportsClean,
  mapping = aes(
    x = Year,
    y = TotalPassengers,
```

```
    color = Airport,
    shape = Location
  )
) +
  geom_point(size = 2) +
  geom_line() +
  scale_color_manual(values = psuPallete) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    x = "Year",
    y = "Total Passengers",
    color = "Airport",
    shape = "Location"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")
library(patchwork)
library(tidyverse)
library(ggplot2)
library(rvest)

# Monte Carlo simulation function
monte_carlo_sim <- function(n, xmin, xmax, ymin, ymax) {
  x <- runif(n, xmin, xmax)
  y <- runif(n, ymin, ymax)
  df <- data.frame(x = x, y = y)
  return(df)
}

# Function to wrangle data and calculate estimate for a given n
run_mc <- function(n) {
  monte_carlo_sim(n, xmin = 0, xmax = 1, ymin = 0, ymax = 1.5) |>
    mutate(
      flag = case_when(
        y <= dbeta(x, shape1 = 2, shape2 = 2) ~ "on/below",
        y > dbeta(x, shape1 = 2, shape2 = 2) ~ "above"
      )
    )
}

# Function to calculate estimate
get_estimate <- function(df) {
```

```r
  df |>
    summarize(
      proportion = sum(flag == "on/below") / n(),
      estimate = proportion * 1.5
    ) |>
    pull(estimate)
}

# Function to make a single plot

psuPallete <- c("#1E407C", "#BC204B", "#3EA39E", "#E98300", "#999999", "#AC8DCE", "#F2665E",

make_plot <- function(df, n) {
  est <- round(get_estimate(df), 4)

  ggplot(df, aes(x = x, y = y, color = flag)) +
    geom_point(size = 0.5, alpha = 0.6) +
    stat_function(
      fun = dbeta,
      args = list(shape1 = 2, shape2 = 2),
      xlim = c(0, 1),
      color = "black",
      linewidth = 0.8
    ) +
    scale_color_manual(values = psuPallete) +
    labs(
      title = paste("Monte Carlo Integration, n =", n),
      x = "x",
      y = "y",
      color = "flag",
      caption = paste("Est. Numerical Integration:", est)
    ) +
    theme_minimal() +
    theme(legend.position = "bottom")
}

# Generate data for each resolution
set.seed(123)
df10    <- run_mc(10)
df100   <- run_mc(100)
df1000  <- run_mc(1000)
df10000 <- run_mc(10000)
```

```r
# Make individual plots
plot10    <- make_plot(df10, 10)
plot100   <- make_plot(df100, 100)
plot1000  <- make_plot(df1000, 1000)
plot10000 <- make_plot(df10000, 10000)

plot10 + plot100 + plot1000 + plot10000
calciumRaw <- read_csv("calcium.csv")

calciumClean <- calciumRaw |>
  setNames(c("y0_null", "y1_null", "y2_null", "y3_null",
             "y0_diet", "y1_diet", "y2_diet", "y3_diet")) |>
  pivot_longer(
    cols = everything(),
    names_to = c("timePoint", "treatment"),
    names_sep = "_"
  ) |>
  mutate(
    timePoint = case_when(
      timePoint == "y0" ~ "Initial",
      timePoint == "y1" ~ "Year 1",
      timePoint == "y2" ~ "Year 2",
      timePoint == "y3" ~ "Year 3"
    ),
    treatment = case_when(
      treatment == "null" ~ "Null",
      treatment == "diet" ~ "Diet/Exercise"
    ),
    value = as.numeric(value)
  ) |>
  rename(measurement = value) |>
  filter(!is.na(measurement))
ggplot(
  data = calciumClean,
  mapping = aes(
    x = timePoint,
    y = measurement,
    color = treatment,
    group = treatment
  )
) +
  geom_point(alpha = 0.4) +
```

```
  geom_smooth(se = FALSE) +
  labs(
    x = "Time Point",
    y = "Bone Density Measurement",
    color = "Treatment"
  ) +
  theme_minimal()
knitr::include_graphics("ClaudePlot1.png")
knitr::include_graphics("ClaudePlot2.png")
```